

Experiment No: 8 Experiment Name: Demonstration of predictive models such as Decision Tree, Neural network and K-Nearest Neighbor using various domain datasets

Aim:

To implement and compare predictive models — Decision Tree, Artificial Neural Network (ANN), and K-Nearest Neighbor (KNN) — for classification tasks using standard domain datasets.

Algorithm:

Step 1: Import libraries (sklearn, tensorflow/keras, pandas, numpy).

Step 2: Load dataset (e.g., Iris for classification).

Step 3: Preprocess: scale features, encode labels, split 80/20.

Step 4: Decision Tree:

- a. Recursively split data on best feature (Gini/Entropy).
- b. Grow tree until stopping criteria is met.
- c. Predict by traversing leaf nodes.

Step 5: K-Nearest Neighbor (KNN):

- a. For each test point, compute distance to all training points.
- b. Select K nearest neighbors.
- c. Predict by majority vote (classification).

Step 6: Neural Network (MLP):

- a. Define layers: Input, Hidden (ReLU), Output (Softmax).
- b. Train using backpropagation and gradient descent.
- c. Predict class probabilities.

Step 7: Evaluate each model: Accuracy, Confusion Matrix.

Step 8: Compare and interpret results.

Program:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load and prepare data
iris = load_iris()
X, y = iris.data, iris.target
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# === Decision Tree ===
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
print("=== Decision Tree ===")
print(f"Accuracy: {accuracy_score(y_test, dt_pred):.4f}")
print(classification_report(y_test, dt_pred, target_names=iris.target_names))

# === K-Nearest Neighbor ===
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_s, y_train)
knn_pred = knn.predict(X_test_s)
print("=== K-Nearest Neighbor (K=5) ===")
print(f"Accuracy: {accuracy_score(y_test, knn_pred):.4f}")
print(classification_report(y_test, knn_pred,
target_names=iris.target_names))

# === Neural Network (MLP) ===
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu',
max_iter=500, random_state=42)
mlp.fit(X_train_s, y_train)
mlp_pred = mlp.predict(X_test_s)
print("=== Neural Network (MLP) ===")
print(f"Accuracy: {accuracy_score(y_test, mlp_pred):.4f}")
print(classification_report(y_test, mlp_pred,
target_names=iris.target_names))

```

Output:

=== Decision Tree ===

Accuracy: 1.0000

=== K-Nearest Neighbor (K=5) ===

Accuracy: 1.0000

=== Neural Network (MLP) ===

Accuracy: 1.0000

All three models achieved 100% accuracy on the Iris test set.

Precision, Recall, F1-Score = 1.00 for all classes (setosa, versicolor, virginica).

Result:

All three predictive models — Decision Tree, KNN, and Neural Network — achieved 100% accuracy on the Iris dataset test set. This demonstrates their effectiveness for multi-class classification. Neural Network provides better generalization for complex datasets, while Decision Tree offers interpretability.